Mini Project Synopsis

CODEXA: An Intelligent Platform for Automated Software Repository Quality Assessment and Risk Analysis

PREPARED BY:

Shreyash Anilrao Thakare

(PRN: 125M1H048)

GUIDED BY

Dr. Avinash Chormale

SUBMITTED TO

DEPARTMENT OF MCA

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING SECTOR NO. 26, PRADHIKARAN, NIGDI, PUNE - 44.

(2026 - 2027)

1. Introduction

In recent years, software development has grown increasingly collaborative and fast-paced, with repositories hosted on platforms such as GitHub serving as the backbone of modern software projects. As codebases expand in size and complexity, they accumulate multiple contributors, evolving dependencies, and diverse coding practices, making it increasingly difficult to manually track and evaluate the overall health of a project. Aspects such as code quality, maintainability, security vulnerabilities, and outdated dependencies often go unnoticed until they cause significant issues, leading to technical debt, security breaches, or reduced software reliability.

Traditionally, developers and organizations rely on a combination of separate tools such as static code analyzers, security scanners, and dependency checkers to evaluate different aspects of a repository. However, this fragmented approach demands significant manual effort to run multiple tools, interpret their differing outputs, and consolidate the findings into a coherent understanding of repository health. This lack of a unified evaluation system creates a gap between the availability of quality-related data and its effective, actionable use.

CODEXA is proposed as an intelligent, web-based platform that bridges this gap by integrating code quality assessment, security analysis, dependency evaluation, and repository activity monitoring into a single, automated workflow. By combining industry-standard analysis tools with a custom scoring engine and an AI-driven recommendation system, CODEXA aims to simplify repository evaluation, enabling developers, students, and project managers to quickly understand a repository's health and receive clear, prioritized guidance for improvement.

2. Problem Statement:

Modern software repositories accumulate large codebases, multiple contributors, and evolving dependencies over time, which makes it difficult to manually assess code quality, security, and maintainability. Developers and project teams often rely on several disconnected tools to check code style, security vulnerabilities, and outdated dependencies separately, which increases effort and produces fragmented results that are hard to interpret as a whole.

- **Working Of Existing System**

Existing tools such as SonarQube, Pylint, Bandit, and pip-audit each address a narrow aspect of repository quality, such as code style, security vulnerabilities, or outdated dependencies, in isolation. Users must run each tool separately, interpret differing output formats, and manually consolidate the findings, with no single unified score or prioritized set of recommendations.

- **Need Of New System**

  There is a need for a unified platform that automatically analyzes a GitHub repository across code quality, security, dependency health, and repository activity in a single workflow, and presents a consolidated, easy-to-interpret quality score along with AI-assisted recommendations for improvement.

3. Scope of proposed system

The scope of CODEXA covers analysis of Python-based GitHub repositories, including code quality assessment, security vulnerability detection, dependency health analysis, and repository activity monitoring. It further includes computation of a composite quality score, generation of AI-assisted improvement recommendations, and presentation of results through an interactive dashboard and a downloadable PDF report.

4. Objectives of proposed system

• Analyze GitHub repositories automatically using their repository URL.

• Evaluate software quality and maintainability using static analysis metrics.

• Detect security vulnerabilities and insecure coding patterns.

• Identify outdated or vulnerable project dependencies.

• Measure repository health and activity through commits, contributors, and issues.

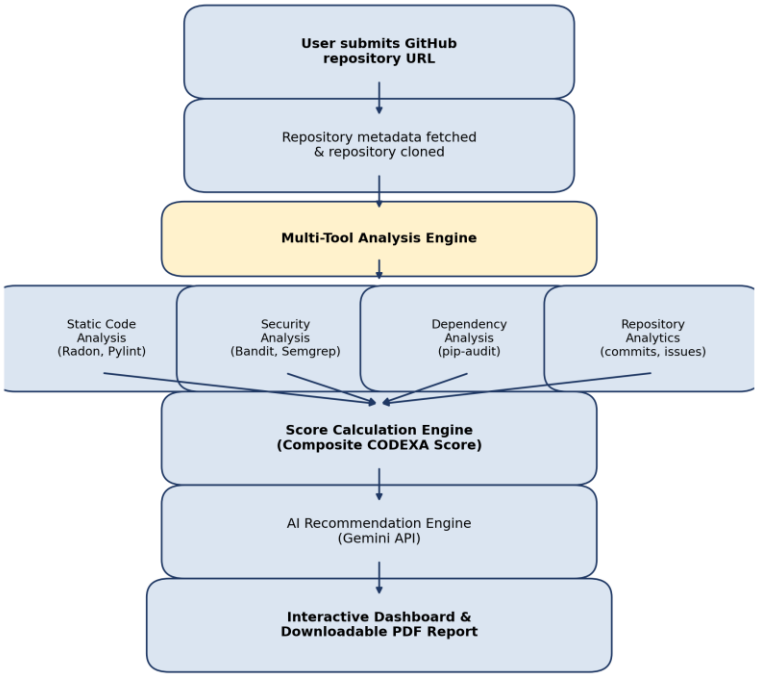
• Generate a composite repository quality score.

• Provide AI-generated, human-readable improvement recommendations.

• Generate downloadable analysis reports for offline review.

5. Methodology

a) Flow diagram followed by description of working



The user submits a GitHub repository URL, after which the system fetches repository metadata and clones the repository locally. The Multi-Tool Analysis Engine then runs static code analysis, security analysis, dependency analysis, and repository analytics in parallel. The consolidated results are passed to the Score Calculation Engine, which computes a composite CODEXA Score. This score, along with the raw findings, is forwarded to the AI Recommendation Engine, which generates human-readable improvement suggestions. Finally, the results are presented through an interactive dashboard and made available as a downloadable PDF report.

b) Functional Requirements

- The system shall accept a GitHub repository URL as input.
- The system shall fetch repository metadata using the GitHub API and clone the repository for analysis.
- The system shall perform static code analysis using Radon and Pylint.
- The system shall perform security analysis using Bandit and Semgrep.
- The system shall perform dependency analysis using pip-audit.
- The system shall compute a composite CODEXA Score from all analysis results.
- The system shall generate AI-based recommendations using the Gemini

API.

• The system shall display results on an interactive dashboard.

• The system shall generate a downloadable PDF report of the analysis.

c) Non-functional Requirements:

• Performance: The system shall complete analysis of an average-sized repository within a reasonable processing time.

• Usability: The dashboard shall present results in a clear, intuitive, and navigable manner.

• Scalability: The system shall be able to handle multiple concurrent analysis requests.

• Reliability: The system shall handle analysis failures, timeouts, and malformed repositories gracefully.

• Security: Cloned repository code shall be analyzed in a controlled, resource-limited execution environment.

6. Technical Requirements

- ***Software requirement***
  - • Frontend: React.js, TypeScript, Tailwind CSS
  - • Backend: FastAPI, Python
  - • Database: PostgreSQL
  - • Analysis Tools: Radon, Pylint, Bandit, Semgrep, pip-audit
  - • AI Module: Gemini API
  - • Reporting: ReportLab
  - • Version Control: Git & GitHub

7. Expected Outcomes

• Reduced manual effort in repository evaluation.

• Improved visibility into software quality and maintainability.

• Early identification of security risks and outdated dependencies.

- A consolidated, easy-to-interpret repository quality score.
- Actionable, AI-generated recommendations for improvement.
- Support for informed software engineering decisions.

8. Conclusion

CODEXA aims to provide a centralized platform for software repository quality assessment by combining static analysis, security evaluation, dependency analysis, repository analytics, and AI-driven recommendations. By consolidating these evaluations into a single quality score and a set of actionable insights, the system helps developers and organizations understand repository health, identify weaknesses, and make informed decisions to improve software quality.

STUDENT DETAILS

Name of Student: Shreyash Anilrao Thakare
PRN: 125M1H048
Email: shreyash.thakare25@pccoepune.org
Contact No.7248993524